Code Signing — Step by Step

Windows (Azure) & macOS (Apple), and how to reuse both for your other projects

APRS Command · James Rospopo (KE4CON)

Follow-along maintainer guide · keep private (accounts & credentials)

What this is

A slow, click-by-click guide to signing your installers so users see **your verified name** as the publisher instead of an "unknown publisher — are you sure?" warning. No prior experience assumed; every term is explained the first time it appears.

Where you are right now:

- **Windows / Azure** — ✓ Identity validation complete. Remaining: create the certificate profile, make the "robot login," and paste six values into GitHub. ~20 minutes.
- **macOS / Apple** — membership paid. Remaining: create the certificate and an API key on a Mac, prove it works by hand once, then paste six values into GitHub.

> **The automation is already built.** Both signing steps live in the repo's release workflow (`.github/workflows/release.yml`) and stay *asleep* until you add the secrets. Nothing signs — and nothing can break — until you flip the switch. Unsigned builds keep shipping exactly as before in the meantime.

What "code signing" actually is

When you download a program, Windows and macOS ask: "*Do we know who made this, and has it been tampered with?*" Code signing answers both. You get a **certificate** — a cryptographic ID card issued after a trusted authority checks who you are — and your build process stamps your installer with it. The operating system checks the stamp and, if it trusts the issuer, shows **your name** instead of a scary warning.

- **Windows** trusts the stamp directly. We use **Azure Artifact Signing** (formerly "Trusted Signing").
- **macOS** needs one extra step, **notarization**: you send the signed app to Apple, their servers scan it, and send back a "ticket" you attach to the app. We use an **Apple Developer ID** certificate plus notarization.

They are completely independent — doing one doesn't affect the other.

Part 1 — Windows (Azure Artifact Signing)

Cost: about **$9.99/month** (the "Basic" plan). **Users see:** your verified legal name as publisher.

Vocabulary: **Azure** = Microsoft's cloud. **Subscription** = your billing container. **Resource group** = a folder that holds related things. **Service principal** = a limited "robot login" you give to GitHub instead of your personal password. **Certificate profile** = the template your signing certificates are minted from.

> **You already did the hard part.** The identity validation (Microsoft's ID check via your phone) is complete. The three remaining pieces are quick.

Step 1 — Create the certificate profile

The profile is the template your signing certificates come from. (Skip if you already made one.)

1. Open **portal.azure.com** → search **Artifact Signing Accounts** → click your account (e.g. `aprscommandsign`).
2. Left menu → under **Objects** → **Certificate profiles** → **+ Create** → choose **Public Trust**.
3. **Certificate profile name:** e.g. `aprscommand-public` (5–100 letters/numbers).
4. **Verified CN and O:** select the identity validation you already completed.
5. Leave the rest default → **Create**.

Step 2 — Copy the three names GitHub needs

Name	Where to find it	Example
Endpoint	Your account's region URI	`https://wus2.codesigning.azure.net/`
Account name	The Artifact Signing account name	`aprscommandsign`
Certificate profile name	From Step 1	`aprscommand-public`

Common region endpoints: West US 2 → `https://wus2.codesigning.azure.net/` · East US → `https://eus.codesigning.azure.net/` · West Europe → `https://weu.codesigning.azure.net/`

Step 3 — Create the "robot login" (service principal)

1. Go back to your **Artifact Signing account** page → left menu → **Settings** → **Properties** (or the **JSON View** link). Copy the long **Resource ID** — it looks like `/subscriptions/1111…/resourceGroups/rg-signing/providers/Microsoft.CodeSigning/codeSigningAccounts/aprscommandsign`
2. At the top of the portal, click the **>_** icon (**Cloud Shell**). First time: choose **Bash** and accept the small storage account.
3. At the `$` prompt, paste this — replace the `--scopes` value with the Resource ID you copied:

```
az ad sp create-for-rbac \
  --name "aprs-command-signing-ci" \
  --role "Trusted Signing Certificate Profile Signer" \
  --scopes "<PASTE-THE-RESOURCE-ID-HERE>"
```

It prints a small block:

```
{
  "appId":    "aaaaaaaa-....",
  "password": "abc123~....",
  "tenant":   "bbbbbbbb-...."
}
```

> **Copy `appId`, `password`, and `tenant` right now.** The `password` is shown **only this once**. If you miss it, just run the command again — it makes a fresh one.

Step 4 — Flip the switch in GitHub

1. Go to **github.com/KE4CON/APRS-Command** → **Settings** → left menu **Secrets and variables** → **Actions**.
2. On the **Secrets** tab, **New repository secret** — add these three (Name, Value, Add secret):

Secret name	Value
`AZURE_TENANT_ID`	the `tenant` from Step 3
`AZURE_CLIENT_ID`	the `appId` from Step 3
`AZURE_CLIENT_SECRET`	the `password` from Step 3

Then the **Variables** tab → **New repository variable** — add these three:

Variable name	Value
`AZURE_SIGNING_ENDPOINT`	your endpoint, e.g. `https://wus2.codesigning.azure.net/`
`AZURE_SIGNING_ACCOUNT`	your account name — **this is the on-switch**
`AZURE_SIGNING_CERT_PROFILE`	your profile name

Step 5 — Test it

Create a throwaway tag (in GitHub Desktop or on the site), e.g. `v0.5.1-test`. Watch the **Actions** tab — the "*Sign installer (Azure Artifact Signing)*" step should **run**, not say "skipped." Delete the draft release and tag afterward.

> GOOD TO KNOW Azure signing usually removes the SmartScreen warning immediately because it chains to a Microsoft-trusted authority. Occasionally the warning briefly returns when Azure rotates to a new back-end authority that hasn't built reputation yet — uncommon, not something you control.

Part 2 — macOS (Apple Developer ID + Notarization)

You paid the **$99/year** membership. Everything here is done **on a Mac**. More moving parts than Windows, but each is small.

Vocabulary: **Developer ID Application certificate** = the ID card that signs apps for distribution outside the Mac App Store. **Keychain Access** = the built-in Mac app that stores certificates/keys. **Notarization** = Apple scans your signed app and returns a ticket. **Stapling** = attaching that ticket so it's trusted offline. **Hardened runtime** = a stricter mode Apple requires; it blocks a few things .NET needs unless you allow them (the "entitlements" file — already in the repo at `build/entitlements.mac.plist`). **.p12** = your certificate + private key in one password-protected file. **.p8** = the API key used to log in to notarization. **base64** = turning a binary file into plain text so it can be pasted into a GitHub secret.

Step 1 — Find your Team ID

1. Go to **developer.apple.com/account** → sign in → **Membership details**.
2. Copy your **Team ID** — 10 characters like `AB12CD34EF`. Your full signing name will be: `Developer ID Application: James Rospopo (AB12CD34EF)`

Step 2 — Create the Developer ID Application certificate (easiest via Xcode)

1. Install **Xcode** from the Mac App Store (large; let it finish).
2. Xcode → **Settings…** → **Accounts** → **+** → **Apple ID** → sign in.
3. Select your team → **Manage Certificates…** → **+** → **Developer ID Application**.
4. It appears in the list. The certificate and its **private key** are now in your login keychain.

No-Xcode alternative: Keychain Access → Certificate Assistant → *Request a Certificate…* (save to disk) → upload the request at **developer.apple.com/account/resources/certificates** → choose **Developer ID Application** → download the `.cer` → double-click to install.

Step 3 — Export the certificate as a `.p12`

1. Keychain Access → **login** keychain → **My Certificates**.
2. Find **Developer ID Application: James Rospopo (…)**. Expand the triangle — you should see a **private key** underneath. (No key = the cert was made on a different Mac; redo Step 2 here.)
3. Hold ⌘ and select **both** the certificate row and the private key row.
4. Right-click → **Export 2 items…** → format **Personal Information Exchange (.p12)** → save as `DeveloperID.p12`.
5. Set a password (this becomes the `APPLE_CERT_PASSWORD` secret). Enter your Mac login password if asked → **Allow**.
6. Turn it into text for GitHub (Terminal):

```
base64 -i ~/Desktop/DeveloperID.p12 | pbcopy
```

Nothing prints — that's normal; it copies the encoded cert to your clipboard. Re-run it right before pasting so it's fresh.

Step 4 — Create an App Store Connect API key (for notarization)

1. **appstoreconnect.apple.com** → **Users and Access** → **Integrations** tab → **App Store Connect API** → **Team Keys**.
2. **+** (Generate API Key) → name it `notarization` → Access **Developer** → **Generate**.
3. **Download the `AuthKey_XXXXXXXXXX.p8` now** — Apple lets you download it only once.
4. Note two IDs on that page: the **Key ID** (10 chars, also in the filename) and the **Issuer ID** (a long UUID near the top).
5. Encode it (Terminal): `base64 -i ~/Downloads/AuthKey_XXXXXXXXXX.p8 | pbcopy`

Step 5 — Prove it works by hand (once)

In the repo folder on your Mac, build the app bundle, then sign / notarize / staple. This confirms your identity string and key before we rely on the automation.

```
bash scripts/make-macos-app.sh osx-arm64

ID="Developer ID Application: James Rospopo (AB12CD34EF)"   # your Team ID
APP="artifacts/installers/APRS Command.app"

# Sign inner files first, main program next, the bundle LAST. Never use --deep.
find "$APP/Contents/MacOS" -name "*.dylib" -exec \
  codesign --force --options runtime --timestamp \
    --entitlements build/entitlements.mac.plist --sign "$ID" {} \;
codesign --force --options runtime --timestamp \
  --entitlements build/entitlements.mac.plist --sign "$ID"
"$APP/Contents/MacOS/Aprs.Desktop"
codesign --force --options runtime --timestamp \
  --entitlements build/entitlements.mac.plist --sign "$ID" "$APP"
codesign --verify --strict --verbose=2 "$APP"

# Notarize (uses your Step 4 key), then staple the ticket.
ditto -c -k --keepParent "$APP" "APRSCommand.zip"
xcrun notarytool submit "APRSCommand.zip" \
  --key ~/Downloads/AuthKey_XXXXXXXXXX.p8 \
  --key-id <KEY_ID> --issuer <ISSUER_ID> --wait
xcrun stapler staple "$APP"

# Repackage the stapled app into the .dmg (does NOT rebuild it unsigned), then sign the
.dmg.
bash scripts/make-macos-app.sh osx-arm64 --dmg-only
codesign --force --timestamp --sign "$ID" "artifacts/installers/APRSCommand-osx-arm64.dmg"
```

If notarization says **Invalid**, get the reason: `xcrun notarytool log <submission-id> --key … --key-id … --issuer …`. Almost always a missing `--options runtime`, an unsigned file, or a missing entitlement.

Step 6 — Add the six Mac secrets to GitHub

The CI job is **already written and dormant** — it self-gates on `APPLE_SIGNING_IDENTITY`, so builds stay unsigned until you add these (GitHub → Settings → Secrets and variables → Actions → **Secrets**):

Secret	What to paste
`APPLE_SIGNING_IDENTITY`	`Developer ID Application: James Rospopo (TEAMID)` — the on-switch
`APPLE_CERT_P12_BASE64`	the base64 from Step 3
`APPLE_CERT_PASSWORD`	the `.p12` password from Step 3
`APPLE_API_KEY_P8_BASE64`	the base64 from Step 4
`APPLE_API_KEY_ID`	the Key ID from Step 4
`APPLE_API_ISSUER_ID`	the Issuer ID from Step 4

Then push a throwaway tag (e.g. `v0.5.1-test`) and confirm the "*Sign + notarize + staple*" step runs.

> **Apple gotchas (each has cost people an evening):**
>
> - Sign inner files first, the bundle last. **Never** `--deep`.
> - `--options runtime` **and** `--timestamp` are both mandatory.
> - The `.p8` key downloads only once. Lose it → revoke and make a new one.
> - Staple the app **before** packaging the `.dmg`, or offline users still get a warning.
> - The Developer ID cert lasts ~5 years, but the **$99 membership is annual** — if it lapses, signing/notarization stop.

Part 3 — Reusing both for your other projects

Short answer: **yes — you set up the accounts once and reuse them for every project.** You do *not* pay again, re-validate your identity, or make new certificates per project. You reuse the same credentials and only wire up each new project's build.

Apple — one membership signs everything

- Your **$99/year membership** covers **all** your apps — no per-app fee.
- The **same Developer ID Application certificate** (the same `.p12`) signs any number of apps.

- The **same App Store Connect API key** (the same `.p8`, Key ID, Issuer ID) notarizes any app.
- Your **Team ID** and the **signing identity string** are the same everywhere.

To sign a new macOS project:

1. Add the same six `APPLE_*` secrets to that project's GitHub repo (copy the exact same values).
2. Add the sign+notarize+staple job to that project's release workflow — copy the `installer-macos` job from APRS Command's `release.yml` and change only the app-specific bits: the app/bundle name and the build-output paths.
3. Give each app its **own unique bundle identifier** in its `Info.plist` — e.g. `com.ke4con.aprs-command`, `com.ke4con.fieldcommand`. The certificate doesn't care what the bundle ID is; it just has to be unique per app.
4. The entitlements file is reusable as-is **for .NET/Avalonia apps**. A different framework (e.g. Electron, native) may need different entitlements — but for your .NET apps it's identical.

> Reality: signing a second .NET macOS app is a copy-paste of the CI job plus the same six secrets. Fifteen minutes, not an afternoon.

Azure — one signing account signs everything

- Your **identity validation** is done once and covers the account — never repeated.
- One **Artifact Signing account** + one **certificate profile** can sign binaries for any number of your projects.
- The **same service principal** (the same `appId` / `password` / `tenant`) works for every repo. (You can make a separate one per project if you prefer isolation, but you don't have to.)
- You pay for the **signing plan** (~$9.99/mo Basic), shared across all projects. Basic includes a generous monthly signing quota — plenty for personal release cadence.

To sign a new Windows project:

1. Add the same three secrets (`AZURE_TENANT_ID` / `AZURE_CLIENT_ID` / `AZURE_CLIENT_SECRET`) and three variables (`AZURE_SIGNING_ENDPOINT` / `AZURE_SIGNING_ACCOUNT` / `AZURE_SIGNING_CERT_PROFILE`) to that project's repo — the exact same values.
2. Copy the "*Sign installer (Azure Artifact Signing)*" step from APRS Command's `release.yml` into that project's workflow, and point `files-folder` at where that project puts its `.exe`.

> TIME-SAVER If you keep several projects, put a **GitHub Organization** around them and add each secret/variable **once at the organization level** (Settings → Secrets and variables → Actions → *New organization secret*), shared to all repos. Then a new project inherits the credentials automatically and you skip re-adding six values each time. (Personal accounts don't have org-level secrets — you add them per-repo; a free org fixes that.)

Per-project checklist (either platform)

Reused as-is (set up once)	New per project
• Apple membership, Developer ID cert (`.p12`), API key (`.p8`), Team/Key/Issuer IDs • Azure account, certificate profile, identity validation, service principal • The six `APPLE_*` / six Azure secret+variable *values* • The .NET entitlements file	• Add the secrets/variables to the new repo (or inherit from a GitHub org) • Copy the signing job/step into the new repo's workflow • Adjust app name, bundle identifier, and build-output paths • Push a `-test` tag to confirm the step runs

---

Security reminder: real certificates, keys, passwords, `.p12` / `.p8` files, and secret values belong **only** in GitHub Actions secrets and your personal password manager — **never** committed to a repository. This guide contains names and steps, never values. The full markdown source lives at `docs/release/CODE_SIGNING_SETUP.md`.